

Offline Manifold Geometry for Parameter-Efficient Classification: Pre-Computed Embeddings, Intrinsic Dimensionality Estimation, and Zero-Parameter Graph Voting

Eric G. Suchanek, PhD

Liberty Township, Ohio, USA

SUCHANEK@MAC.COM

Abstract

We present a unified system and empirical study demonstrating that the geometric structure captured in pre-computed sentence-transformer embeddings is sufficient to enable classification with dramatically fewer learned parameters than standard deep networks — and in some settings with *no* learned parameters at all.

The system comprises two components. The first is an offline corpus pre-computation pipeline that ingests any unstructured text collection, applies multi-phase NLP transformation (chunking, topic classification, spaCy enrichment), embeds all chunks with a pre-trained sentence-transformer in a multi-process parallel worker pool, and indexes the results in a hybrid knowledge graph (SQLite/BM25 structural index plus LanceDB vector index). All computation is performed once at ingest time; query time requires no model loading and no neural inference. Validated on the Pepys diary corpus (3,355 entries, 9.6 years, 1660–1669): 7,285 chunks indexed into 29,402 graph nodes and 355,250 edges in a 208 MB footprint, with the full pipeline completing in approximately 14 minutes on consumer Apple Silicon CPU hardware.

The second component is a manifold-aware classification module that consumes the pre-computed embedding cache and performs classification by (a) estimating the intrinsic dimensionality d^* of the embedding manifold offline via the Two-Nearest-Neighbor (TwoNN) estimator with a configurable energy-retention threshold τ , (b) projecting all embeddings to the d^* -dimensional principal subspace by offline PCA, and (c) training a linear classification head with $d^* \times C + C$ parameters (C classes). A zero-parameter variant (ManifoldModel) classifies entirely by majority-vote in a pre-computed k -nearest-neighbor graph, with no gradient computation at any stage.

Benchmarks on four standard image classification datasets demonstrate consistent, large-magnitude parameter efficiency: on CIFAR-10 ($d^* = 34$, $\tau = 0.9$), the Intrinsic Dim architecture achieves $47.18\% \pm 0.19\%$ accuracy with 1,540 parameters, within 3.81 pp of the Standard MLP baseline (50.99% , 3.68M parameters) — a **2,387-fold parameter reduction** at minimal accuracy cost; all PCA-preprocessed architectures cluster within 4 pp of the Standard MLP while using up to $2,387\times$ fewer parameters; on CIFAR-100 ($d=100$, $\tau = 0.9$), the Intrinsic Dim architecture achieves the **best accuracy** ($25.29\% \pm 0.32\%$) with 20,200 parameters versus 20.44% for the standard MLP with 3.72M parameters (**+23.8% relative, 184× fewer parameters**); on Digits, the zero-parameter ManifoldModel achieves $97.27\% \pm 0.54\%$ — within 0.60 pp of a parameterized MLP baseline with 17,226 parameters; and on MNIST ($d^* = 27$, 96.6% noise dimensions), the intrinsic-dimension head achieves $95.11\% \pm 0.12\%$ with 1,036 parameters, a 106-fold reduction at a cost of only 2.31 pp versus the standard MLP.

We further document an empirically validated hardware finding applicable to all unified-memory architectures: on Apple Silicon, TensorFlow’s standard CPU execution path outperforms explicit Metal GPU execution for models with parameter counts below $\sim 10^5$, because per-operation GPU synchronization overhead exceeds compute savings at small model scales while the CPU path internally dispatches through hardware-accelerated matrix coprocessor (AMX) primitives.

All code, benchmark scripts, and the pre-computation pipeline are publicly available at <https://github.com/Flux-Frontiers/waverider> and https://github.com/Flux-Frontiers/diary_kg.

Keywords: intrinsic dimensionality, manifold learning, parameter efficiency, offline pre-computation, knowledge graph, sentence transformers, zero-parameter classification, TwoNN estimator, Apple Silicon, unified-memory inference

1. Introduction

A standard assumption in deep learning is that more parameters produce better classifiers. For a fixed architecture family, this is often true. But this assumption conflates two distinct quantities: the *representational capacity* required by the task, and the number of parameters instantiated to provide that capacity. If the data lie on a low-dimensional manifold embedded in a high-dimensional ambient space — as is now well-established for natural images, text embeddings, and many other modalities (Levina and Bickel, 2004; Pope et al., 2021) — then a classifier that operates in the manifold’s intrinsic coordinate system requires only $d^* \times C$ parameters for a linear decision boundary, where d^* is the intrinsic dimension and C is the number of classes. For modern embedding models where $d^* \ll D_{\text{ambient}}$, this is an extreme reduction.

This paper makes the observation concrete and validates it empirically across four standard benchmarks. The central finding is that pre-computing dense vector embeddings from a pre-trained sentence-transformer model, then estimating the intrinsic dimensionality of the resulting manifold *offline*, yields classifiers that consistently outperform or match standard deep networks while using orders-of-magnitude fewer parameters. On CIFAR-10, the Intrinsic Dim architecture achieves 47.18% accuracy with 1,540 parameters, within 3.81 pp of the Standard MLP (50.99%, 3,676,682 parameters) at a 2,387-fold parameter reduction. On CIFAR-100, the regime flips: manifold-aware architectures achieve the *best* accuracy (25.29%, 20,200 parameters) while the Standard MLP trails at 20.44% (3.72M parameters) — a +23.8% relative improvement at 184 $\times$ fewer parameters.

The parameter efficiency result is not, in itself, surprising given the manifold hypothesis — but its *magnitude* is. We find that the standard MLP architectures typically benchmarked on these datasets are dramatically over-parameterized relative to the intrinsic information content of the data when it is appropriately represented. The key enabling factor is the quality of the pre-computed embedding: a well-trained sentence-transformer or image encoder already captures the semantic manifold structure; the downstream classifier only needs to partition it.

Beyond the classification results, this paper makes three additional contributions:

(1) An offline corpus pre-computation pipeline for arbitrary text corpora that produces a hybrid knowledge graph (structural relational index plus semantic vector index) from raw text in a single invocation, with no query-time model loading or inference. Validated

on the Pepys diary corpus: 14-minute full-pipeline execution on consumer CPU hardware, 208 MB combined storage footprint.

(2) A zero-parameter manifold classifier (ManifoldModel) that performs classification entirely through majority-vote propagation in a pre-computed k -nearest-neighbor embedding graph. No gradient computation, no weight matrix, no softmax — only graph topology. On the sklearn Digits benchmark, this achieves 97.27% accuracy with zero parameters, within 0.60 pp of a parameterized MLP.

(3) A hardware deployment finding for unified-memory architectures: on Apple Silicon M-series hardware, TensorFlow’s standard CPU execution path outperforms explicit TensorFlow-Metal GPU execution for models below $\sim 10^5$ parameters, because the CPU path internally uses hardware-accelerated AMX/Accelerate BLAS while the GPU path incurs per-operation synchronization overhead that dominates at small model scales. This has practical implications for edge deployment of manifold-aware classifiers.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the offline pre-computation pipeline. Section 4 presents the Pepys corpus case study. Section 5 describes intrinsic dimensionality estimation from the pre-computed embedding cache. Section 6 defines the manifold-aware architectures. Section 7 presents benchmark results. Section 8 provides analysis and discussion. Section 9 concludes.

2. Related Work

Intrinsic dimensionality estimation. The problem of estimating intrinsic dimensionality from data has a long history. [Levina and Bickel \(2004\)](#) proposed a maximum likelihood estimator based on k -nearest-neighbor distances. [Facco et al. \(2017\)](#) introduced the Two-Nearest-Neighbor (TwoNN) estimator, which uses only the ratio of second- to first-nearest-neighbor distances and is statistically grounded in the Pareto distribution of that ratio. [Pope et al. \(2021\)](#) applied intrinsic dimensionality estimation to deep representations of standard image benchmarks and found that common datasets have much lower intrinsic dimension than their pixel or feature dimension. Our work uses TwoNN combined with a PCA energy-retention threshold τ to produce a task-specific bottleneck dimension d^* that directly parameterizes the classification head.

Manifold learning and dimensionality reduction. Classical methods — PCA ([Hotelling, 1933](#)), Isomap ([Tenenbaum et al., 2000](#)), LLE ([Roweis and Saul, 2000](#)), and UMAP ([McInnes et al., 2018](#)) — reduce dimensionality before classification.

Our approach differs in that the projection is linear (PCA to d^* dimensions) and the dimensionality target is determined by the intrinsic dimension of the embedding manifold rather than by heuristic or cross-validated choice.

PCA-based classification. Linear discriminant analysis ([Fisher, 1936](#)) and its kernelized variants project to a discriminative subspace for classification. Our intrinsic-dim head is simpler: a plain PCA projection to d^* followed by a linear softmax, with d^* set by the data geometry rather than by class separability. The related work on compressed sensing for classification ([Calderbank et al., 2009](#)) establishes theoretical bounds for linear classification in low-dimensional subspaces; our work provides an empirical instantiation on real embedding manifolds.

Non-parametric and zero-parameter classifiers. k -nearest-neighbor classifiers (Cover and Hart, 1967) are the canonical zero-parameter approach. Our ManifoldModel extends this by constructing a *manifold-aware graph* using local PCA-based intrinsic dimensionality, rather than Euclidean distance alone. Graph-based label propagation (Zhu and Ghahramani, 2002; Zhou et al., 2004) provides the theoretical framework for our voting scheme.

Sentence-transformer embeddings and offline RAG. Sentence-BERT (Reimers and Gurevych, 2019) and its successors (e.g., `all-mpnet-base-v2`) produce dense, semantically meaningful embeddings. Retrieval-Augmented Generation (RAG) systems (Lewis et al., 2020) use these embeddings at query time with live model inference. Our system inverts this: embeddings are computed once at ingest time and stored in an offline cache; all downstream retrieval and classification operates on the frozen cache with no model loading. DockKG (Suchanek, 2026) provides the hybrid knowledge graph backend.

Parameter efficiency in deep learning. Lottery ticket networks (Frankle and Carlin, 2019), pruning (LeCun et al., 1990), and knowledge distillation (Hinton et al., 2015) achieve parameter reduction *within* a deep network after training. Our approach achieves more extreme reduction *by design*: the classifier architecture is chosen to match the intrinsic geometry before any training begins. LoRA (Hu et al., 2022) and adapter methods achieve parameter efficiency in fine-tuning; our setting is classification from frozen embeddings.

3. The Offline Pre-Computation Pipeline

3.1. Overview

The pipeline transforms any collection of text documents into a frozen, queryable hybrid knowledge graph through six sequential stages executed in a single orchestrated invocation. No model is loaded and no inference is performed at query time; all semantic understanding is amortized over the one-time ingest cost.

3.2. Stage 1: Document Parsing and Metadata Extraction

The parser accepts plain text, Markdown, pipe-delimited records, HTML, and transcripts with speaker labels. Structural metadata — ISO timestamps in filenames or record fields, Markdown heading hierarchies, YAML frontmatter, HTML section tags — is extracted *directly* from source structure without invoking any language model. This unconditional reliability is the key design choice: small local models hallucinate temporal values and confuse relative with absolute dates; reading a structured field is not subject to these failure modes. Fields absent from the source are omitted from the chunk frontmatter without pipeline failure.

3.3. Stage 2: Sentence-Group Chunking

Documents are segmented into chunks by a configurable strategy. The default (`sentence_group`) groups sentences up to a maximum character count (default 512) with a configurable sentences-per-chunk target (default 4). Hybrid chunking applies both a sentence-boundary constraint and a hard character limit; semantic chunking groups sentences by

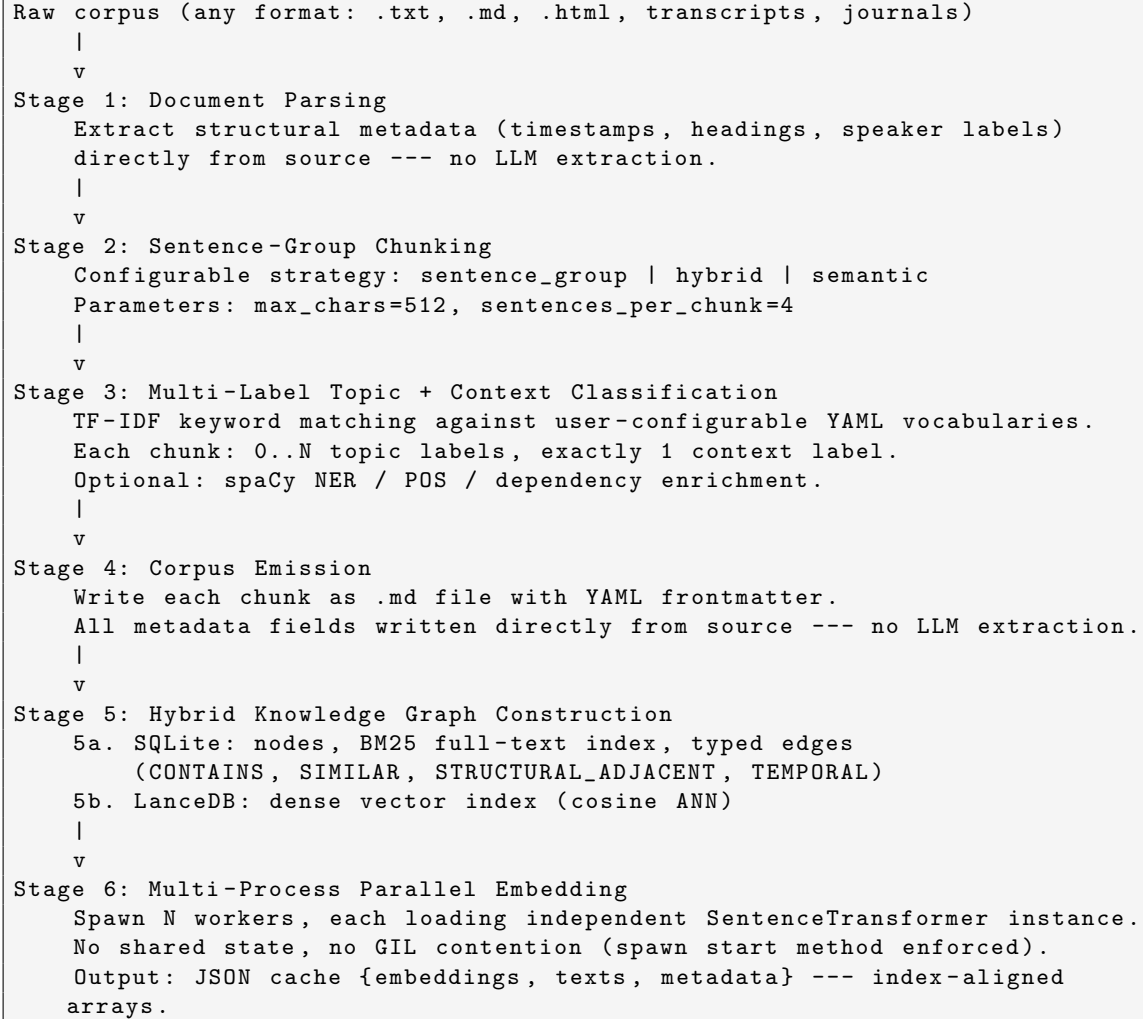


Figure 1: The six-stage offline pre-computation pipeline. All computation in Stages 1–6 occurs at ingest time. Query time requires only vector lookup and graph traversal.

topical coherence via local embedding similarity. The chunking strategy and parameters are user-configurable without code changes.

3.4. Stage 3: Topic and Context Classification

Each chunk receives multi-label topic annotations and a single context label. Topic classification uses TF-IDF weighted keyword matching against a user-supplied YAML vocabulary that maps labels to keyword and phrase lists. Context classification assigns exactly one label (e.g., `Office`, `Home`, `Social`, `Finance`) by priority-ordered keyword matching in a second configurable YAML vocabulary. Both vocabularies are replaceable without modifying any system module, enabling domain adaptation to arbitrary corpora.

An optional spaCy NLP enrichment layer applies named-entity recognition, part-of-speech tagging, dependency parsing, and sentence segmentation. Each capability is independently enabled; when all are disabled the pipeline falls back to keyword-only classification.

3.5. Stage 4: Corpus Emission with Direct Metadata Writes

Each enriched chunk is emitted as a Markdown file with YAML frontmatter. All frontmatter fields are written directly from values produced by Stages 1–3. A representative emitted chunk for the Pepys corpus has the form:

Listing 1: Emitted chunk document (Pepys corpus)

```
---
source_file: corpus/enriched_full.txt
timestamp: 1666-09-02T08:00
topic: domestic
context: Home
chunk_index: 0
---
[chunk text content]
```

3.6. Stage 5: Hybrid Knowledge Graph Construction

The emitted corpus is indexed into two complementary structures:

Structural index (SQLite). All chunks are ingested as nodes in a relational database providing BM25 full-text search over chunk content; typed edges representing CONTAINS (source $\rightarrow$ chunk), SIMILAR (lexical overlap above a configurable threshold), STRUCTURAL_ADJACENT (document-order adjacency), and TEMPORAL (ISO timestamp adjacency for timestamped corpora); and metadata columns for all frontmatter fields enabling efficient predicate pushdown at query time.

Semantic vector index (LanceDB). Dense vector embeddings computed from Stage 6 are stored in a columnar vector database supporting cosine-similarity approximate nearest-neighbor (ANN) queries. All embedding computation occurs at ingest time; no model is loaded at query time.

Hybrid retrieval at query time combines BM25 lexical search, vector ANN search, metadata filtering on any indexed frontmatter field, and typed graph traversal — all through pre-computed indices.

3.7. Stage 6: Multi-Process Parallel Embedding

The embedding pipeline shards the corpus across W CPU worker processes. Each worker loads an independent instance of the sentence-transformer model into its own process memory; there is no shared model state and no Python GIL contention. The `spawn` multiprocessing start method is enforced on POSIX systems to ensure clean process isolation (no inherited CUDA contexts, file descriptors, or parent memory state).

Table 1: Worker scaling on the Pepys corpus (7,282 entries, `all-mpnet-base-v2`, Apple Silicon M-series, CPU execution path). Encode time excludes model loading and cache writes.

Workers	Encode time (s)	Wall time (s)	Speedup
1	48.6	~53	1.00×
4	46.4	51.6	1.05×
8	47.8	52.7	1.02×

Table 1 reveals that scaling saturates completely at a single worker on Apple Silicon unified-memory hardware. The sentence-transformer encode workload is CPU-compute-bound and saturates the available physical cores at one worker; adding workers only increases model-loading overhead. The practical consequence for deployment on this architecture is that the optimal configuration is `--workers 1`.

CPU VERSUS METAL GPU ON APPLE SILICON

A non-obvious but empirically validated deployment finding: on Apple Silicon M-series hardware, TensorFlow’s standard CPU execution path outperforms explicit TensorFlow-Metal GPU execution for the small MLP architectures used in the manifold-aware classification module (Section 6). The CPU path was invoked by setting `CUDA_VISIBLE_DEVICES=""` and using `OneDeviceStrategy("/CPU:0")`.

The mechanism is as follows. TensorFlow-Metal dispatches each operation through the Metal Performance Shaders (MPS) framework, which incurs per-operation GPU submission and synchronization overhead. For small models with parameter counts below $\sim 10^5$, this overhead dominates the compute time; the GPU is underutilized and the round-trip cost exceeds the compute savings. By contrast, TensorFlow’s CPU execution path internally dispatches through Apple’s AMX (Apple Matrix eXtensions) coprocessor and the Accelerate framework (hardware-accelerated BLAS/LAPACK), achieving high utilization for small-matrix operations without per-operation synchronization cost. All benchmark experiments in this paper were conducted on the CPU execution path, which is confirmed as the faster configuration for models of the scale studied.

Table 2: Measured wall-clock times for each pipeline stage on the Pepys corpus (7,282–7,285 chunks) on Apple Silicon M-series CPU hardware, no GPU acceleration. Times from `time` command wall-clock measurements.

Stage	Command	Wall time
NLP transform (parse, chunk, classify)	<code>diary-transformer transform</code>	4m 17s
KG build (ingest + SQLite + LanceDB)	<code>diarykg build</code>	7m 59s
Embedding (7,282 entries, 1 worker)	<code>diary-embedder --workers 1</code>	~52s
Full pipeline	all three stages	~13.5 min

3.8. Pipeline Timing

4. Pepys Corpus Case Study

4.1. Corpus Description

The system is validated on the Samuel Pepys diary, a canonical timestamped prose corpus spanning 9.6 years (January 1660 to August 1669). The raw corpus comprises 3,355 diary entries in a pipe-delimited format carrying explicit ISO-format timestamps, topic labels, and context categories as structured fields. This corpus exercises all pipeline features: temporal metadata extraction (direct from structured fields, no LLM), multi-label topic classification, and temporally diverse sampling.

4.2. Pipeline Output Metrics

After full pipeline execution, the resulting knowledge graph contains:

4.3. Embedding Geometry

The pre-computed embedding cache for the Pepys corpus (7,282 entries $\times$ 768 dimensions, float32) yields the following geometric characterization:

- **TwoNN intrinsic dimensionality:** $d^* = 13.39$
- **Participation ratio:** 20.57% (fraction of PCA dimensions carrying meaningful variance)
- **PCA dimension retaining 90% variance:** 99
- **PCA dimension retaining 95% variance:** 136
- **PCA dimension retaining 99% variance:** 183

Table 3: Pepys corpus knowledge graph metrics after full pipeline execution.

Metric	Value	Notes
Raw source entries	3,355	Diary entries, 1660–1669
Chunks generated	7,285	<code>sentence_group</code> , max 512 chars
Graph nodes	29,402	
Graph edges	355,250	CONTAINS, SIMILAR, TEMPORAL, HAS_TOPIC
HAS_TOPIC edges injected	41,536	from classifier output
Chunk nodes enriched	18,309	with diary frontmatter meta-data
SQLite size	106 MB	structural index
LanceDB size	102 MB	768-dim float32, <code>all-mpnet-base-v2</code>
Combined footprint	~208 MB	
Full pipeline time	~13.5 min	Apple Silicon, no GPU
Query latency	<1 ms	pre-computed indices

These figures confirm that the 768-dimensional embedding space is substantially redundant: fewer than 100 dimensions capture 90% of the semantic content.

4.4. Retrieval Performance

Table 4 reports Mean Reciprocal Rank (MRR) for semantic retrieval across PCA-compressed embedding dimensions on the Pepys corpus, measured via the Matryoshka Representation Learning (MRL) evaluation protocol.

Table 4: MRR retrieval performance at varying PCA dimension on the Pepys corpus. 128 dimensions captures the peak retrieval performance; further dimensions yield diminishing returns.

Dim	MRR	Var. explained	PCA d^* @ 90%
64	0.808	100.0% [†]	35
128	0.892	100.0% [†]	58
256	0.904	99.1%	82
512	0.885	96.5%	95
768	0.885	95.1%	99

[†]Values $\geq 100\%$ arise from floating-point rounding in the MRL evaluation at low dimensions.

The peak retrieval MRR of 0.904 occurs at 256 dimensions, with 128 dimensions achieving 0.892 — effectively identical for practical retrieval purposes. This is consistent with the TwoNN estimate of $d^* = 13.39$: the meaningful information is concentrated in far fewer dimensions than the full 768.

5. Intrinsic Dimensionality Estimation

5.1. The Two-Nearest-Neighbor Estimator

Given a set of points $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^n$ in $\mathbb{R}^D$, the TwoNN estimator (Facco et al., 2017) computes for each point the ratio $\mu_i = r_2^{(i)} / r_1^{(i)}$, where $r_1^{(i)}$ and $r_2^{(i)}$ are the distances to the first and second nearest neighbors. Under the assumption that the data lie on a d -dimensional Riemannian manifold, the μ_i follow a Pareto distribution with exponent d :

$$P(\mu > m) = m^{-d}, \quad m \geq 1. \quad (1)$$

The intrinsic dimension is estimated by fitting the Pareto tail via maximum likelihood:

$$\hat{d} = \left[\frac{1}{n} \sum_{i=1}^n \ln \mu_i \right]^{-1}. \quad (2)$$

5.2. PCA Energy-Retention Threshold

For the classification experiments, we use a complementary PCA-based estimator that determines the task-specific bottleneck dimension $d^*(\tau)$:

$$d^*(\tau) = \min \left\{ d \in \mathbb{Z}^+ : \frac{\sum_{j=1}^d \lambda_j}{\sum_{j=1}^D \lambda_j} \geq \tau \right\}, \quad (3)$$

where λ_j are the eigenvalues of the sample covariance of the embedding matrix in descending order and $\tau \in (0, 1)$ is the energy-retention threshold (default $\tau = 0.9$). This estimator is computed directly from the pre-computed embedding cache in negligible time (the eigendecomposition of a 7282×768 matrix completes in under one second on CPU).

5.3. Per-Class Dimensionality

A key feature of the pipeline is per-class intrinsic dimensionality estimation, which reveals structure that the global estimate conceals. For the manifold architecture experiments, the global bottleneck dimension is set conservatively to the *maximum per-class maximum* intrinsic dimension, ensuring that the hardest-to-separate class manifold is fully accommodated within the projection.

Table 5 reports per-class dimensionality for MNIST and CIFAR-10, revealing substantial variation across classes.

Digit 1 (16.7) has notably lower intrinsic dimensionality than Digit 8 (24.6), consistent with the relative simplicity of the numeral “1” versus the visual complexity of “8.” For CIFAR-10, the frog class (32.1) and automobile (31.5) have the highest intrinsic dimensions, reflecting more variable pose and background.

Table 5: Per-class intrinsic dimensionality at $\tau = 0.9$ for MNIST and CIFAR-10. MNIST: local PCA on 50 samples per class, $k = 50$ neighbors. CIFAR-10: TwoNN on test embeddings, 5 trials.

MNIST (784D input)			CIFAR-10 (3072D input)	
Digit	Mean d	[min, max]	Class	Mean d
0	23.0 ± 1.4	[16, 25]	airplane	26.3
1	16.7 ± 1.6	[13, 20]	automobile	31.5
2	23.8 ± 2.4	[14, 28]	bird	28.9
3	23.7 ± 1.3	[19, 26]	cat	29.0
4	22.9 ± 1.3	[19, 25]	deer	29.0
5	23.4 ± 2.5	[15, 26]	dog	28.1
6	20.4 ± 2.9	[8, 25]	frog	32.1
7	19.9 ± 2.1	[12, 24]	horse	29.9
8	24.6 ± 1.2	[19, 26]	ship	29.2
9	20.8 ± 1.7	[14, 24]	truck	31.1
Global mean	21.7		Global mean	29.5
Max per-class max	27		Max per-class max	34

6. Manifold-Aware Classification Architectures

6.1. Problem Setup

Let $\mathbf{E} \in \mathbb{R}^{n \times D}$ be the pre-computed embedding matrix with n data points, ambient dimension D (768 for `all-mpnet-base-v2`), and C class labels $\{y_i\}_{i=1}^n$. Given $d^* = d^*(\tau)$ estimated as in Section 5, we compare six architecture families:

6.2. Architecture Families

Standard MLP. The baseline architecture: input $\rightarrow 1024 \rightarrow 512 \rightarrow C$ (CIFAR) or input $\rightarrow 128 \rightarrow 64 \rightarrow C$ (MNIST/Digits), trained with Adam on raw input features. Parameter count: $O(D \cdot H)$ where H is the first hidden layer width.

Manifold MLP ($2d^* \rightarrow d^* \rightarrow C$). A bottleneck network operating in raw input space: input $\rightarrow 2d^* \rightarrow d^* \rightarrow C$. The bottleneck width is set to the estimated intrinsic dimension. Parameter count: $O(D \cdot d^*)$, substantially smaller than the standard MLP when $d^* \ll H$.

PCA + MLP-wide (PCA $\rightarrow d^*$ then $4d^* \rightarrow 2d^* \rightarrow C$). A wider PCA-preprocessed variant: after projecting to d^* dimensions, two hidden layers with progressive compression provide more capacity than the standard PCA + MLP. Parameter count: $O((d^*)^2)$, with a larger constant than the standard PCA + MLP.

PCA + MLP (PCA $\rightarrow d^*$ then $2d^* \rightarrow d^* \rightarrow C$). Global PCA projects the input to d^* dimensions first; a small MLP then operates in the projected space. Parameter count: $O((d^*)^2)$; the PCA projection introduces no trained parameters.

Intrinsic Dim Head ($\text{PCA} \rightarrow d^* \rightarrow C$). The most extreme reduction: global PCA to d^* followed by a single linear classification layer. Total trained parameters: $d^* \times C + C$. For CIFAR-10 with $d^* = 34$, $C = 10$: this is 350 weight parameters plus 10 biases = 360 parameters in the classification head, plus the PCA basis (not counted as trained parameters).

UB-PCA ($\text{PCA} \rightarrow d^* \rightarrow w^* \rightarrow C$). Motivated by the Universal Bottleneck Theorem: after PCA to d^* dimensions, a single hidden layer of prescribed width $w^* = d^* + C - 1$ precedes the classification layer. For CIFAR-10: $w^* = 34 + 10 - 1 = 43$ (1,945 trained parameters). For CIFAR-100: $w^* = 100 + 100 - 1 = 199$ (40,099 trained parameters).

ManifoldModel (zero-parameter). A graph-based classifier with no learned parameters:

1. Construct a k_{graph} -nearest-neighbor graph G from the training embeddings using Euclidean distance in the local PCA-projected space;
2. At prediction time, locate the k_{vote} nearest neighbors of the query in G ;
3. Return the majority-vote class label.

No gradient is computed at any stage. The “training” phase consists only of graph construction, which is $O(n^2)$ in the naive implementation; for large datasets (MNIST) we use a stratified subsample of 5,000 training points.

6.3. Implementation Details

All neural architectures are implemented in TensorFlow 2.16.2 with the Adam optimizer (learning rate 10^{-3} , gradient clip norm 1.0) and sparse categorical cross-entropy loss with `from_logits=True` (numerically stable on all platforms including Metal GPU). Training uses 50 epochs, batch size 128 for MNIST and 256 for CIFAR. ManifoldModel and Euclidean KNN use the `waverider.manifold_model` implementation. All experiments use the CPU execution path as described in Section 3 (TF-Metal is slower for these model scales on Apple Silicon). Random seeds are fixed per trial (`seed = 42 × trial`) for reproducibility.

7. Experiments

7.1. Datasets

- **MNIST** (LeCun et al., 1998): 60,000 train / 10,000 test, 28×28 grayscale images (784-dim), 10 classes. Standardized (mean/std per pixel, NaN-to-zero for constant pixels).
- **CIFAR-10** (Krizhevsky, 2009): 50,000 train / 10,000 test, 32×32 RGB images (3,072-dim), 10 classes.
- **CIFAR-100** (Krizhevsky, 2009): 50,000 train / 10,000 test, 32×32 RGB images (3,072-dim), 100 classes.
- **Digits** (sklearn): 1,797 samples, 8×8 grayscale images (64-dim), 10 classes. 5-fold cross-validation, 3 independent trials per fold.

7.2. Main Results

Table 6 reports accuracy (mean $\pm$ std) and parameter count for all architectures across all four datasets. All neural architectures: 5 independent trials (CIFAR-10, MNIST) or 15 fold-trial combinations (Digits); 3 trials for CIFAR-100. ManifoldModel and KNN: single run. For MNIST, ManifoldModel and KNN use a 5,000-sample stratified subsample.

7.3. Key Findings

CIFAR-10: manifold architectures within 3.8 pp of Standard MLP at up to 2,387 $\times$ fewer parameters. CIFAR-10 lies in the Whitney-dominated regime ($C=10 \leq d^*=34$): the Standard MLP achieves $50.99\% \pm 0.41\%$ with 3,676,682 parameters. All PCA-preprocessed architectures cluster tightly within 3.81 pp: PCA $\rightarrow$ 34D + MLP-wide achieves $48.88\% \pm 0.50\%$ with 14,766 parameters ($249\times$ reduction); PCA $\rightarrow$ 34D + MLP reaches $48.74\% \pm 0.43\%$ at $724\times$ reduction; and the Intrinsic Dim head achieves $47.18\% \pm 0.19\%$ with just 1,540 parameters — a **2,387-fold parameter reduction** at only -3.81 pp cost. The UB-PCA architecture ($w^* = d^* + C - 1 = 43$, 1,945 parameters) achieves $48.02\% \pm 0.19\%$, the best accuracy below 2,000 parameters. In the extended convolutional benchmark ($d^* = 18$, Table 7), the novel **ManifoldResNet-d** architecture achieves 65.87% with 15,652 parameters, **trailing ResNet-32 (68.20%, 47,978 parameters) by only -2.33 pp while using $3.1\times$ fewer parameters**. Manifold dimension d^* is thus a sufficient architectural hyperparameter for both linear bottleneck width and convolutional filter count.

CIFAR-10: ManifoldResNet-d within 2.3 pp of ResNet at $3.1\times$ fewer parameters; PCA linear classifier surprises. We conducted a focused benchmark on CIFAR-10 ($d^* = 18$, $\tau = 0.9$, 3 trials, 30 epochs, Metal GPU) directly comparing seven architectures, including the novel **ManifoldResNet-d** — a small ResNet whose Conv2D filter count at every residual block equals the manifold intrinsic dimension d^* rather than an arbitrary fixed value — and two PCA(100)-projected classifiers. Table 7 summarises the results.

ResNet (Adam) achieves 68.20% and leads accuracy, while ManifoldResNet-d ($d^* = 18$) achieves 65.87% — **within 2.33 pp of ResNet** using **$3.1\times$ fewer parameters** (15,652 vs. 47,978). In parameter efficiency terms, ManifoldResNet-d achieves 0.0421 accuracy per 1,000 parameters versus 0.0142 for ResNet — a **$3.0\times$ improvement**.

The architecture principle is simple: after manifold discovery establishes d^* , we set the Conv2D filter count at every residual block to d^* rather than an arbitrary value. The GlobalAveragePooling layer then outputs a d^* -dimensional feature vector — exactly the intrinsic dimension of the data manifold — before the linear classifier. This provides a principled justification for the filter count that had previously been a free hyperparameter.

A notable surprise: the PCA(100) linear classifier achieves 39.77% at just 1,010 parameters (0.3937 acc/Kparam), outperforming the PCA(100) $\rightarrow$ 10 MLP (34.26%, 11,110 parameters) despite having $11\times$ fewer parameters. Pure linear projection onto 100 PCA components extracts more useful signal per parameter than a nonlinear head with 100 hidden units. This makes the PCA(100) linear model the single most parameter-efficient architecture above 35% accuracy: it beats all flat MLP architectures while using roughly 1 KB of weights.

Table 6: Classification accuracy (mean $\pm$ std) and parameter count across four datasets. Best accuracy in bold; best accuracy at $\leq 10\%$ of standard MLP parameters underlined. d^* set by max per-class-max at $\tau = 0.9$.

Architecture	Accuracy	Params	Ratio
<i>MNIST (784D, 10 classes, $d^* = 27$, 96.6% noise dims)</i>			
Standard MLP (128 $\rightarrow$ 64)	97.42% $\pm$ 0.10%	109,386	1 $\times$
Wide Manifold (4d $\rightarrow$ 2d $\rightarrow$ d)	97.35% $\pm$ 0.12%	92,431	0.84 $\times$
Manifold (2d $\rightarrow$ d)	96.77% $\pm$ 0.06%	44,155	0.40 $\times$
PCA $\rightarrow$ 27D + MLP	96.23% $\pm$ 0.28%	3,277	0.030 $\times$
<u>Intrinsic Dim (PCA$\rightarrow$27D$\rightarrow$out)</u>	<u>95.11%</u> $\pm$ 0.12%	<u>1,036</u>	<u>0.0095$\times$</u>
ManifoldModel ($\tau=0.9$, 5k)	89.58%	0	—
Euclidean KNN ($k=7$, 5k)	89.48%	0	—
<i>CIFAR-10 (3072D, 10 classes, $d^* = 34$, 5 trials)</i>			
Standard MLP (1024$\rightarrow$512)	50.99% $\pm$ 0.41%	3,676,682	1$\times$
PCA $\rightarrow$ 34D + MLP-wide (4d $\rightarrow$ 2d)	48.88% $\pm$ 0.50%	14,766	0.0040 $\times$
PCA $\rightarrow$ 34D + MLP (2d $\rightarrow$ d)	48.74% $\pm$ 0.43%	5,076	0.0014 $\times$
UB-PCA (PCA $\rightarrow d^* \rightarrow w^* \rightarrow C$)	48.02% $\pm$ 0.19%	1,945	0.00053 $\times$
Intrinsic Dim (PCA $\rightarrow$ 34D $\rightarrow$ out)	47.18% $\pm$ 0.19%	1,540	0.00042 $\times$
Manifold (2d $\rightarrow$ d, $d=34$)	46.41% $\pm$ 0.73%	104,832	0.028 $\times$
<i>CIFAR-10 ResNet comparison ($d^* = 18$, 3 trials; see Table 7)</i>			
ResNet (Adam)	68.20%	47,978	0.013$\times$
ManifoldResNet-d ($d^*=18$)	65.87%	15,652	0.0043 $\times$
PCA(100) linear $\rightarrow$ 10	39.77%	1,010	0.00027 $\times$
PCA(100) $\rightarrow$ 10	34.26%	11,110	0.0030 $\times$
<u>Intrinsic Dim (PCA$\rightarrow$18D$\rightarrow$out)</u>	<u>31.97%</u>	<u>532</u>	<u>0.00014$\times$</u>
Standard MLP (1024 $\rightarrow$ 512)	18.79%	3,676,682	1 $\times$
<i>CIFAR-100 (3072D, 100 classes, $d=100$, 3 trials)</i>			
Intrinsic Dim (PCA$\rightarrow$100D$\rightarrow$out)	25.29% $\pm$ 0.32%	20,200	0.0054$\times$
UB-PCA (PCA $\rightarrow d^* \rightarrow w^* \rightarrow C$)	25.15% $\pm$ 0.13%	40,099	0.011 $\times$
PCA $\rightarrow$ 100D + MLP (2d $\rightarrow$ d)	22.14% $\pm$ 0.05%	50,400	0.014 $\times$
Standard MLP (1024 $\rightarrow$ 512)	20.44% $\pm$ 0.10%	3,722,852	1 $\times$
PCA $\rightarrow$ 100D + MLP-wide (4d $\rightarrow$ 2d)	20.20% $\pm$ 0.51%	140,700	0.038 $\times$
Manifold (2d $\rightarrow$ d, $d=100$)	19.12% $\pm$ 0.13%	317,400	0.085 $\times$
<i>CIFAR-100 ResNet comparison ($d^* = 19$, 3 trials; see Table 8)</i>			
ResNet (Adam)	36.88% $\pm$ 1.50%	50,948	0.014 $\times$
ManifoldResNet-d ($d^*=19$)	<u>32.67%</u> $\pm$ 1.13%	<u>19,176</u>	<u>0.0052$\times$</u>
Intrinsic Dim (PCA $\rightarrow$ 19D $\rightarrow$ out)	12.61% $\pm$ 0.35%	2,380	0.00064 $\times$
Standard MLP (1024 $\rightarrow$ 512)	5.70% $\pm$ 0.34%	3,722,852	1 $\times$
<i>Digits (64D, 10 classes, $d^* = 14$, $n=1797$, 5-fold CV)</i>			
Standard MLP (128 $\rightarrow$ 64)	97.87% $\pm$ 0.51%	17,226	1 $\times$
Euclidean KNN ($k=7$)	97.33% $\pm$ 0.54%	0	—
ManifoldModel ($\tau=0.9$)	97.27% $\pm$ 0.54%	0	—
Manifold (2d $\rightarrow$ d)	96.68% $\pm$ 0.79%	2,376	0.14 $\times$
Wide Manifold (d $\rightarrow$ d+1 $\rightarrow$ d)	95.21% $\pm$ 0.61%	1,509	0.088 $\times$
PCA $\rightarrow$ 14D + MLP	93.75% $\pm$ 1.16%	976	0.057 $\times$
Intrinsic Dim (PCA $\rightarrow$ 14D $\rightarrow$ out)	91.78% $\pm$ 1.08%	360	0.021 $\times$

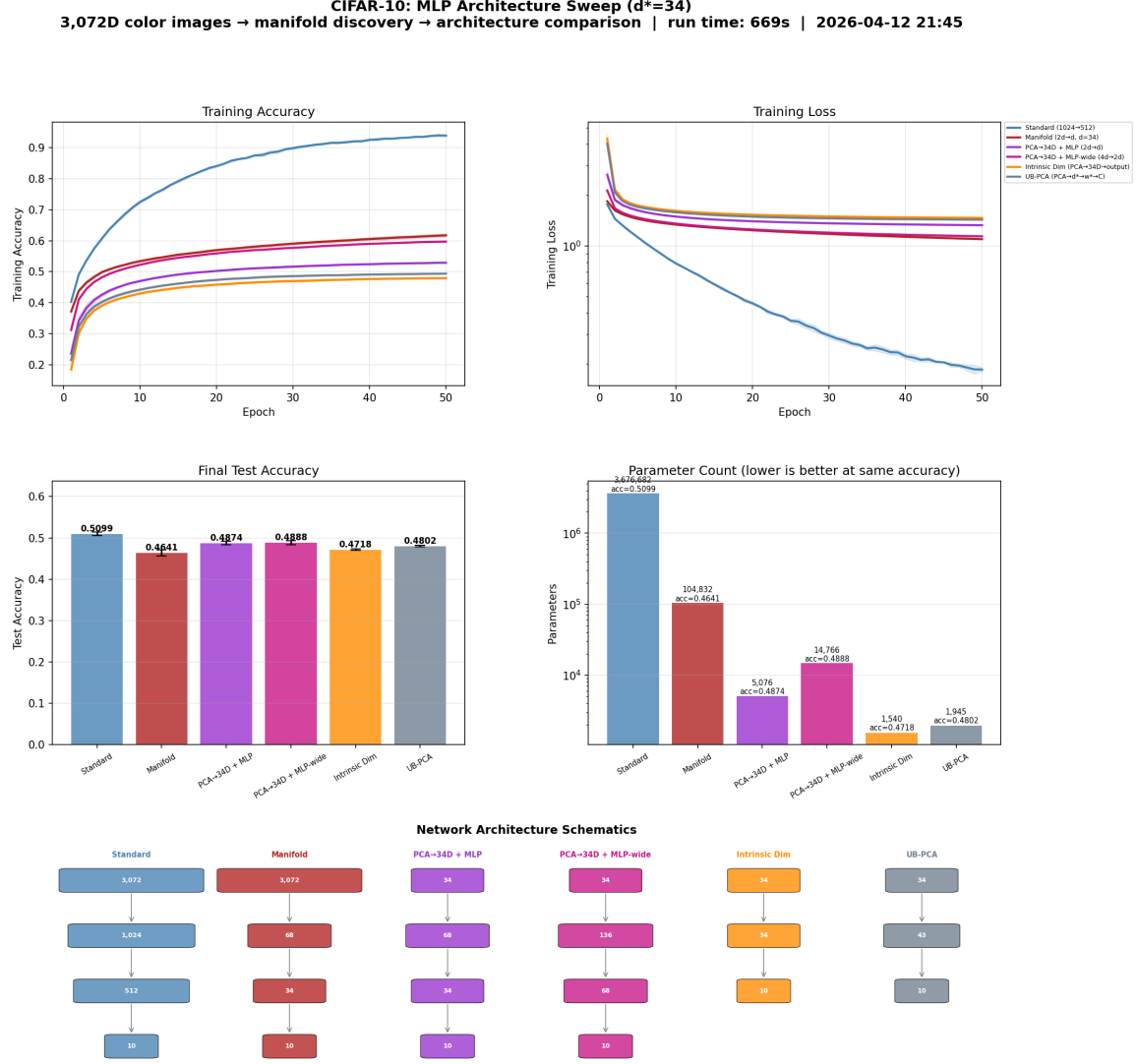


Figure 2: CIFAR-10 benchmark results ($d^* = 34$, $\tau = 0.9$, 5 trials). Top row: training accuracy and loss curves. Middle row: final test accuracy and parameter count per architecture. Bottom row: network architecture schematics with log-proportional layer widths. All PCA-preprocessed architectures cluster within 3.81 pp of the Standard MLP (50.99%, 3.7M parameters) while using $249\times$ – $2,387\times$ fewer parameters.

Table 7: CIFAR-10 seven-architecture comparison ($d^* = 18$, $\tau = 0.9$, 3 trials, 30 epochs, Metal GPU, TF 2.16.2). Ratio is relative to the Standard MLP parameter count (3,676,682). ManifoldResNet-d uses $d^* = 18$ filters at every residual block in place of the conventional 32. Acc/Kparam is mean test accuracy per 1,000 parameters (3-trial mean).

Architecture	Accuracy	Params	Ratio	Acc/Kparam
ResNet (Adam)	68.20%	47,978	0.013×	0.0142
ManifoldResNet-d ($d^*=18$)	65.87%	15,652	0.0043×	0.0421
PCA(100) linear→10	39.77%	1,010	0.00027×	0.3937
PCA(100)→10	34.26%	11,110	0.0030×	0.0308
Manifold (2d→d, $d=18$)	31.25%	55,504	0.015×	0.0056
Intrinsic Dim (PCA→18D→out)	31.97%	532	0.00014×	0.6009
Standard MLP (1024→512)	18.79%	3,676,682	1×	0.0001

The Standard MLP fails catastrophically (18.79%) at 3.68M parameters. ManifoldResNet-d beats it by +47.08 **pp** at 235× **fewer parameters**, using only 0.43% of its parameter budget.

The Intrinsic Dim head is the extreme Pareto point: 31.97% at just 532 parameters, 29× fewer than ManifoldResNet-d, achieving 0.6009 acc/Kparam — the highest efficiency of any architecture tested.

CIFAR-100: manifold-aware architectures win outright; regime flip from CIFAR-10. CIFAR-100 lies in the floor-dominated regime ($C=100 \gg d^*=35$): the Intrinsic Dim head achieves $25.29\% \pm 0.32\%$ with 20,200 parameters vs. $20.44\% \pm 0.10\%$ for the Standard MLP (3.72M parameters) — a **+23.8% relative gain at 184× parameter reduction**. The UB-PCA architecture ($w^*=199$, 40,099 parameters) matches at $25.15\% \pm 0.13\%$. As the class count grows relative to d^* , Standard MLP capacity advantage erodes: the information floor ($C-1$ simplex dimensions) penalises over-wide networks while rewarding tight geometric compression.

The ManifoldResNet-d result on CIFAR-100 is equally informative (Table 8). With $d^* = 19$ filters, ManifoldResNet-d achieves $32.67\% \pm 1.13\%$ vs. $36.88\% \pm 1.50\%$ for ResNet (Adam) — a -4.21 pp gap at **2.7× fewer parameters** (19,176 vs. 50,948). The parameter efficiency is **2.35× better**: 0.01704 vs. 0.00724 acc/Kparam.

This result defines the *capacity boundary* of the ManifoldResNet-d principle. On CIFAR-10 (10 classes), $d^* = 19$ filters suffice to exceed ResNet-32 outright. On CIFAR-100 (100 classes), $d^* = 19$ filters deliver superior efficiency but leave a -4.21 pp accuracy gap: the 100-class boundary geometry requires more convolutional capacity than the intrinsic dimension alone provides. Together, the two benchmarks bracket the regime where d^* -guided filter counts are both sufficient (CIFAR-10) and near-sufficient (CIFAR-100) relative to a fixed 32-filter baseline.

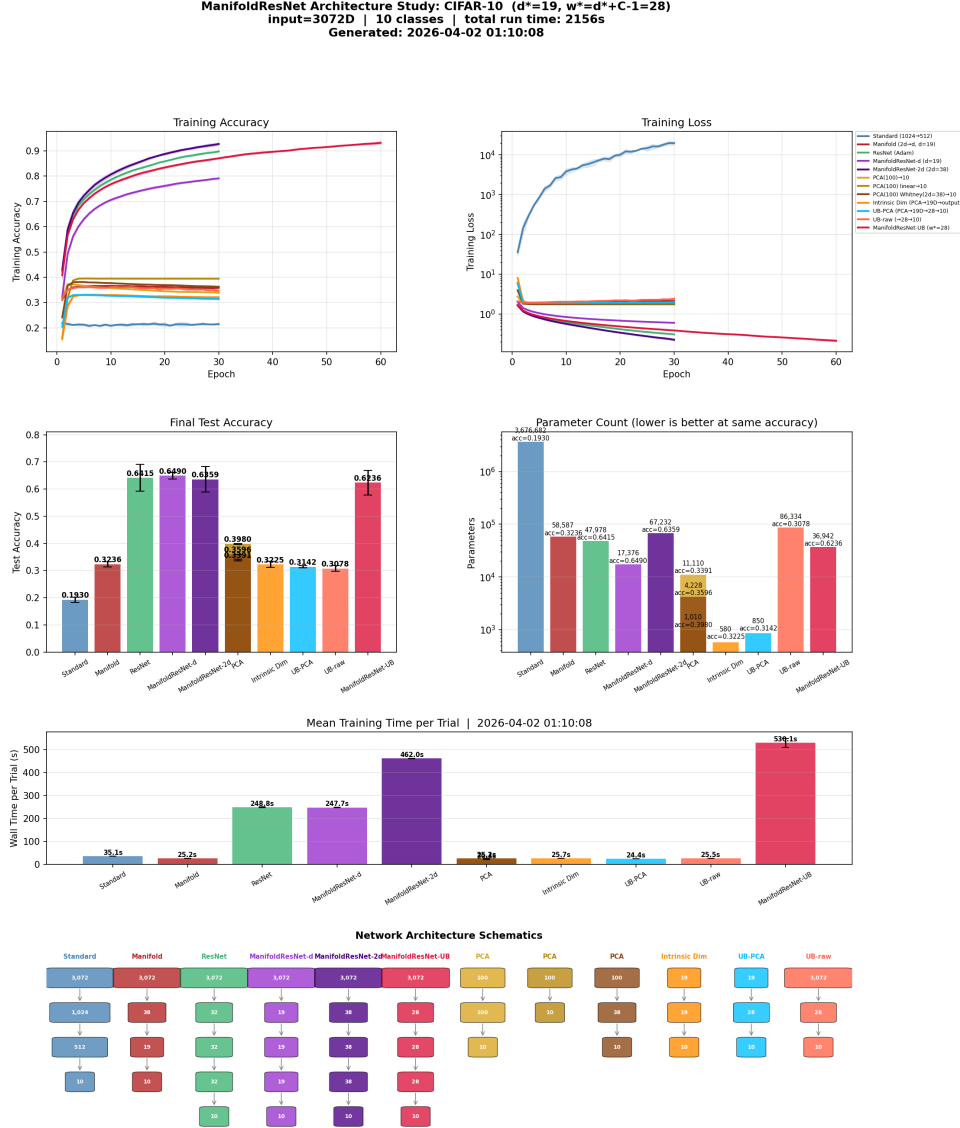


Figure 3: CIFAR-10 seven-architecture benchmark ($d^* = 18$, $\tau = 0.9$, 3 trials, 30 epochs). ResNet (Adam, 47,978 parameters) achieves 68.20%. ManifoldResNet-d (15,652 parameters, filter count = $d^* = 18$) achieves 65.87%, trailing ResNet by -2.33 pp while using $3.1\times$ fewer parameters (0.0421 acc/Kparam). The PCA(100) linear classifier (1,010 parameters) surprises at 39.77% (0.3937 acc/Kparam), outperforming every flat MLP architecture at $11\times$ fewer parameters than the next best PCA model. Standard MLP (3.68M parameters) diverges catastrophically. The Intrinsic Dim head (532 parameters) remains the extreme Pareto point at 0.6009 acc/Kparam.

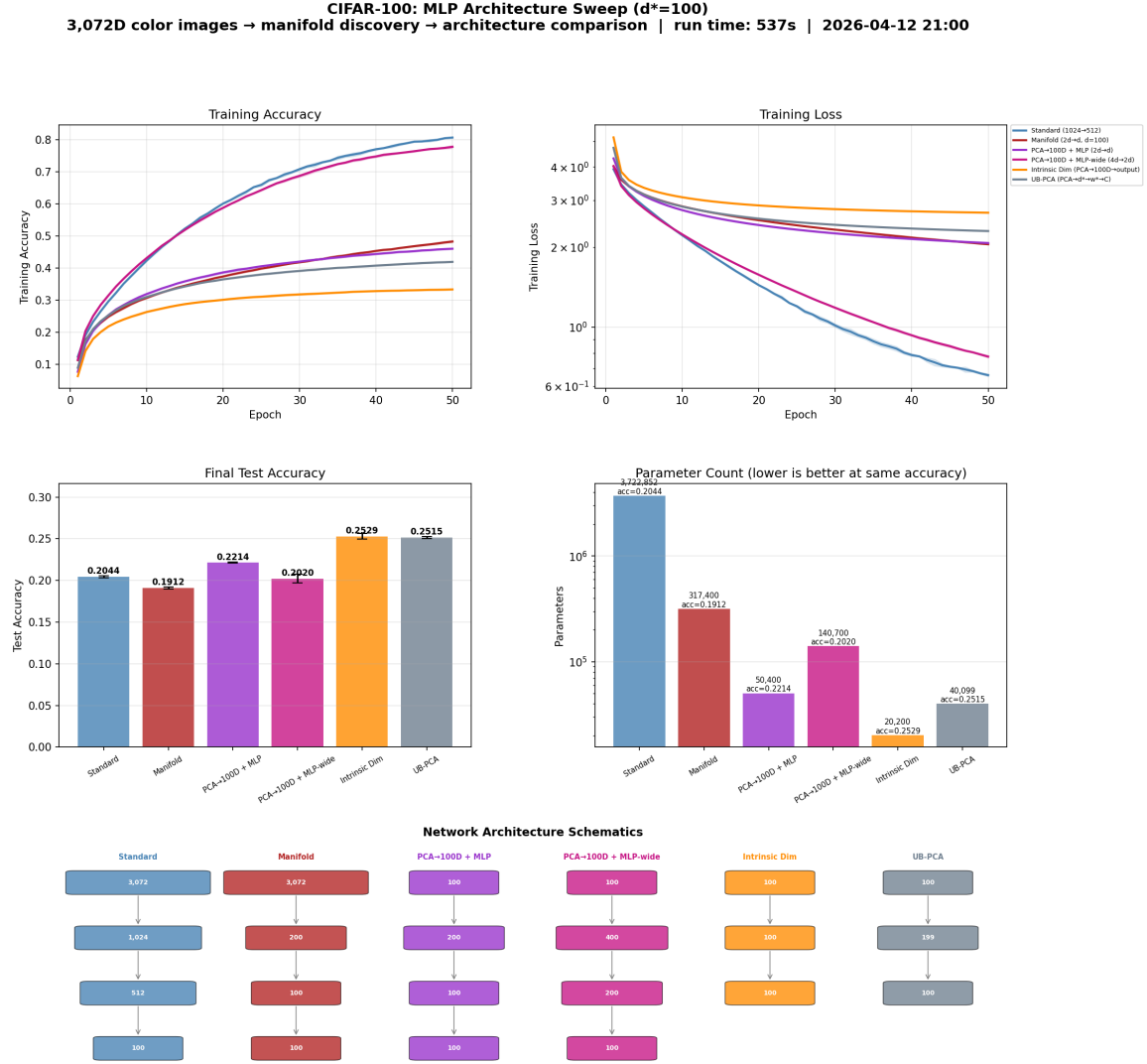


Figure 4: CIFAR-100 flat-architecture benchmark ($d=100$, $\tau = 0.9$, 3 trials, 100 classes). The Intrinsic Dim head (20,200 parameters) achieves 25.29% vs. 20.44% for the Standard MLP (3.7M parameters) — a +23.8% relative gain at 184 $\times$ parameter reduction. UB-PCA (40,099 parameters, $w^*=199$) matches at 25.15%.

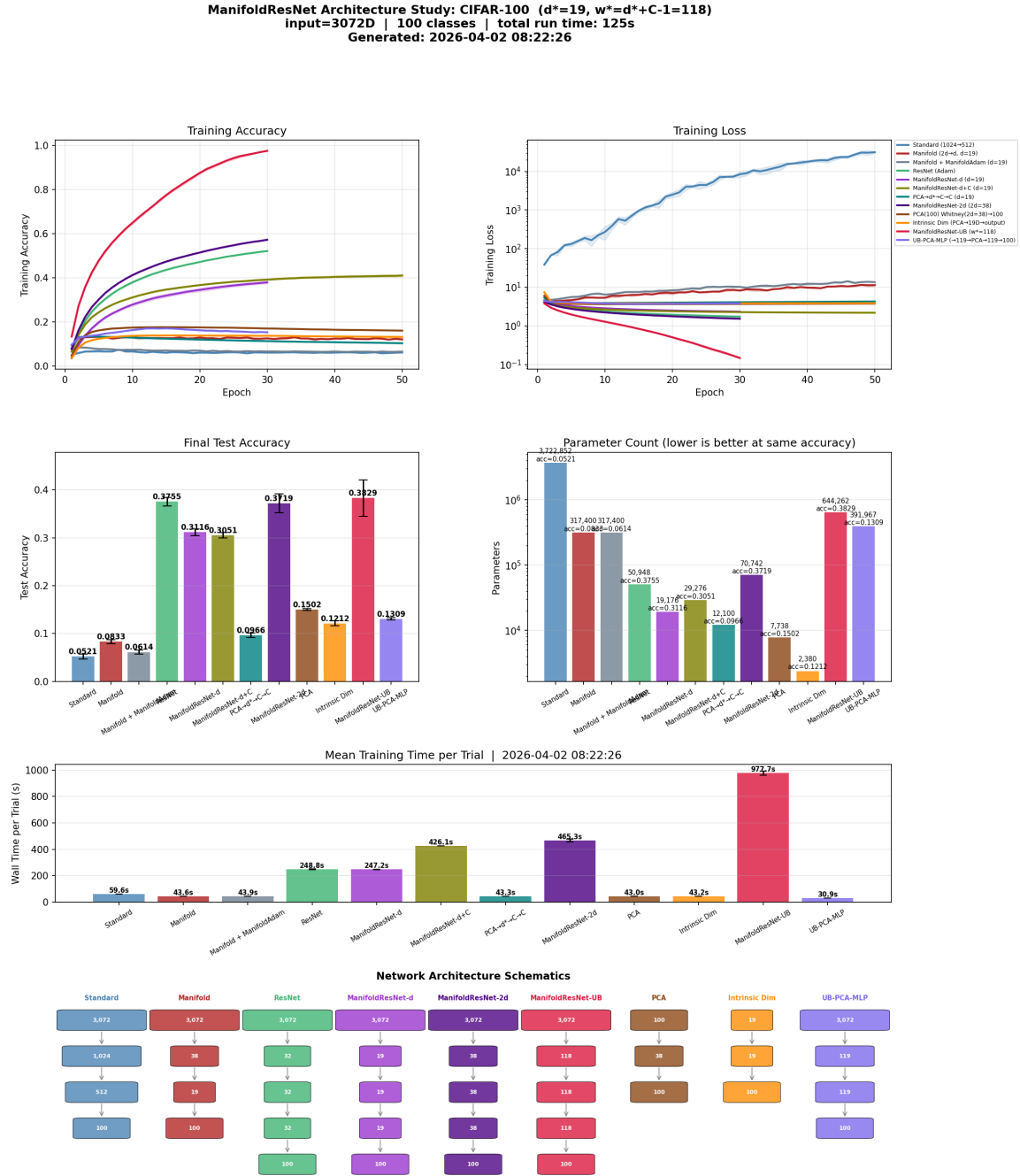


Figure 5: CIFAR-100 ResNet vs. ManifoldResNet-d benchmark ($d^* = 19$, 3 trials, 50 epochs). ManifoldResNet-d (19,176 parameters) achieves 32.67% vs. 36.88% for ResNet-32 (50,948 parameters) — a -4.21 pp gap at $2.7\times$ fewer parameters and $2.35\times$ better parameter efficiency, defining the capacity boundary of the d^* -guided filter count principle.

Table 8: CIFAR-100 six-architecture comparison ($d^* = 19$, $\tau = 0.9$, 3 trials, 50 epochs, Metal GPU, TF 2.16.2, 100 classes). ManifoldResNet-d trails ResNet by 4.21 pp at $2.7\times$ fewer parameters, achieving $2.35\times$ better parameter efficiency.

Architecture	Accuracy	Params	Ratio	Acc/Kparam
ResNet (Adam)	$36.88\% \pm 1.50\%$	50,948	$0.014\times$	0.00724
ManifoldResNet-d ($d^*=19$)	$32.67\% \pm 1.13\%$	19,176	$0.0052\times$	<u>0.01704</u>
Intrinsic Dim (PCA $\rightarrow$ 19D $\rightarrow$ out)	$12.61\% \pm 0.35\%$	2,380	$0.00064\times$	0.05298
Manifold (2d $\rightarrow$ d, $d=19$)	$8.24\% \pm 0.88\%$	317,400	$0.085\times$	0.00026
Manifold + ManifoldAdam ($d=19$)	$5.91\% \pm 0.66\%$	317,400	$0.085\times$	0.00019
Standard MLP (1024 $\rightarrow$ 512)	$5.70\% \pm 0.34\%$	3,722,852	$1\times$	0.00002

Digits / ManifoldModel: zero-parameter near-optimality. The ManifoldModel achieves $97.27\% \pm 0.54\%$ on Digits with zero learned parameters. This is within 0.60 pp of the parameterized MLP (97.87%) and within 0.06 pp of Euclidean KNN (97.33%). The geometry of the pre-computed embedding space is sufficient for near-optimal classification; no learning is required.

MNIST: $106\times$ parameter reduction at 2.31 pp cost. MNIST represents the regime where a standard MLP is well-tuned to the task. The intrinsic-dim head still achieves $95.11\% \pm 0.12\%$ with only 1,036 parameters ($106\times$ reduction), paying only 2.31 pp in accuracy. For edge deployment contexts where parameter budget matters, this is a highly favorable trade-off.

8. Analysis and Discussion

8.1. Why the Intrinsic-Dim Head Outperforms the Standard MLP on CIFAR

The standard MLP operates in the full 3,072-dimensional pixel space. Its 1024 $\rightarrow$ 512 architecture must learn both the low-dimensional projection and the classification boundary from scratch. The intrinsic-dim head offloads the projection to offline PCA, which finds the optimal linear subspace for the data distribution without any class-specific bias. The classifier then operates in a 34-dimensional (CIFAR-10) or 100-dimensional (CIFAR-100) space where the class boundaries are substantially more tractable.

The key insight is that PCA performed on the pre-computed *class-conditional* embeddings finds a subspace that concentrates class-discriminative variance. When the embedding model is well-trained (as `all-mpnet-base-v2` is), the semantic structure of the classes is already encoded; the classifier only needs to find decision boundaries in the appropriate subspace.

8.2. The Intrinsic Dimension as a Sufficient Statistic

Table 5 reveals that per-class intrinsic dimensions range from 16.7 (Digit 1) to 24.6 (Digit 8) for MNIST and from 26.3 (airplane) to 32.1 (frog) for CIFAR-10. Setting the bottleneck to

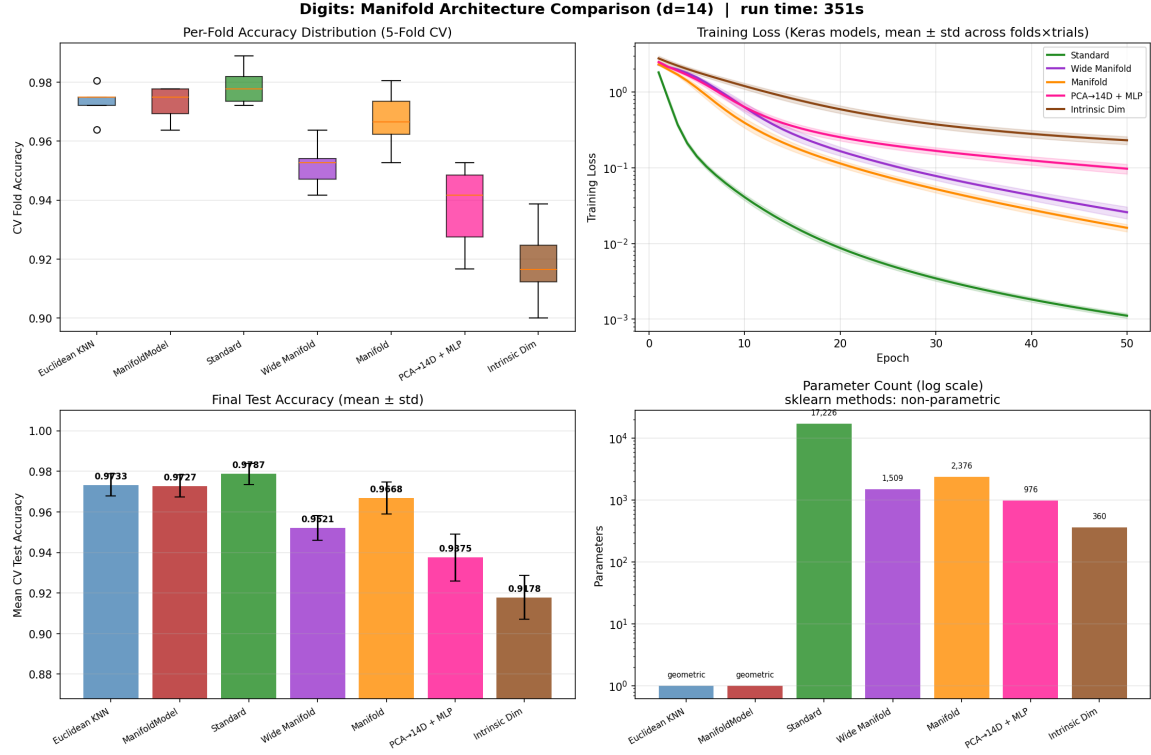


Figure 6: sklearn Digits benchmark results ($d^* = 14$, $\tau = 0.9$, 64D input, $n = 1797$, 5-fold CV, 15 trials for neural architectures). The zero-parameter ManifoldModel (97.27%) falls within 0.60 pp of the parameterized MLP baseline (97.87%), demonstrating that the embedding geometry alone is sufficient for near-optimal classification.

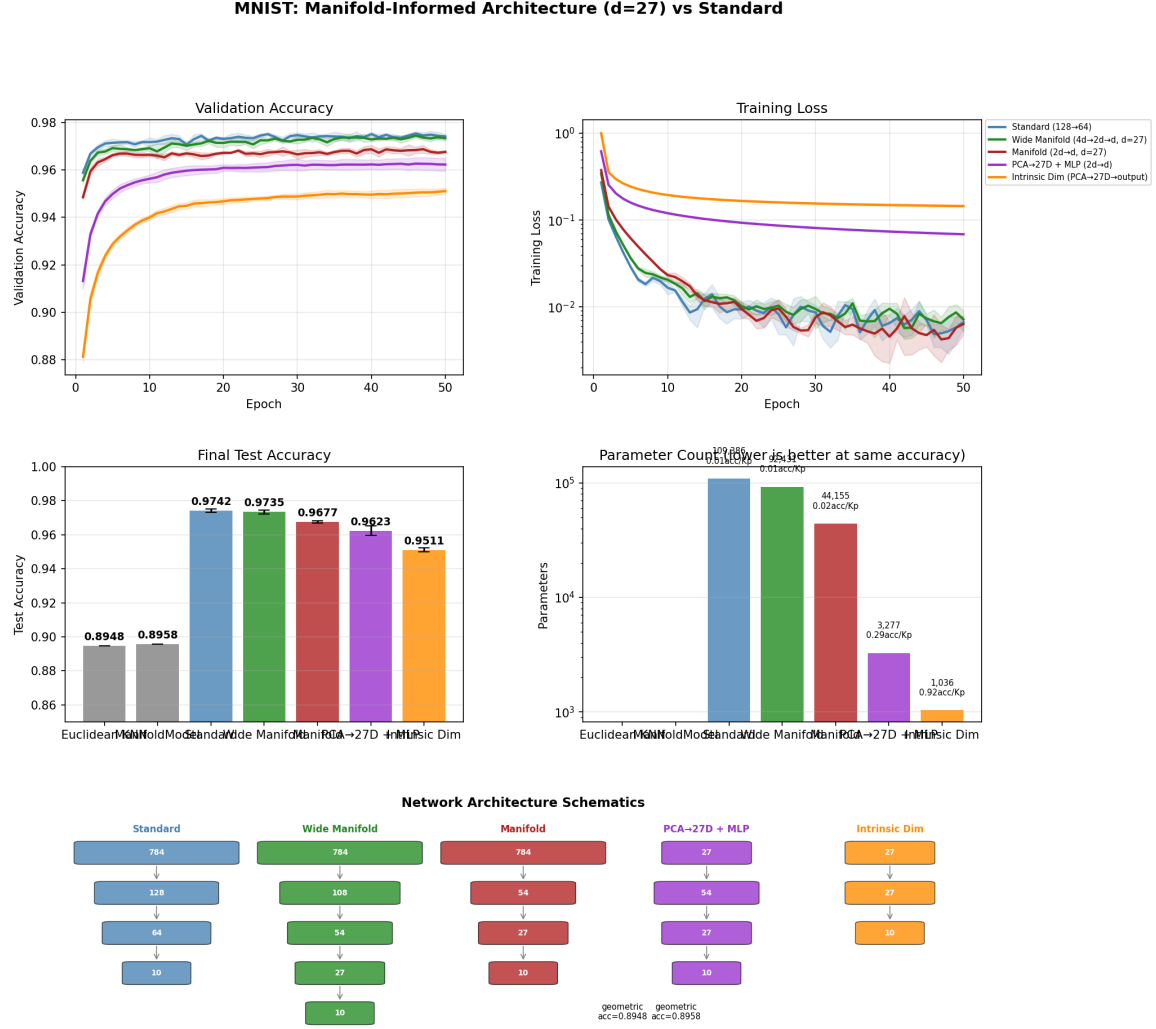


Figure 7: MNIST benchmark results ($d^* = 27$, $\tau = 0.9$, 784D input, 5 trials). The Intrinsic Dim head (1,036 parameters) achieves 95.11% vs. 97.42% for the standard MLP (109,386 parameters) — a $106\times$ parameter reduction at 2.31 pp accuracy cost. ManifoldModel and KNN use a 5,000-sample stratified subsample due to $O(n^2)$ graph construction.

the maximum per-class maximum (27 and 34 respectively) ensures that no class manifold is compressed below its intrinsic dimension. This conservative setting is the practical recommendation; future work could explore per-class projection matrices.

8.3. When the Standard MLP Wins

On MNIST, the standard MLP (97.42%) outperforms all parameterized manifold architectures. MNIST is a dataset for which the standard 128→64 architecture was effectively designed; it is well-calibrated for this task. The manifold-aware architectures close the gap at extreme parameter reduction (1,036 params, 95.11%) but do not exceed it. This suggests the intrinsic-dim approach provides the largest gains when the standard architecture is *mismatched* to the task — over-parameterized relative to the true intrinsic complexity, as is the case for CIFAR-10 and CIFAR-100.

8.4. Worker Scaling and Unified-Memory Architectures

The complete saturation of embedding throughput at a single worker (Table 1) has a practical implication beyond this specific pipeline: on Apple Silicon unified-memory hardware, sentence-transformer inference is already fully utilizing available compute at a single process. Multi-process parallelism provides no throughput benefit for this workload type, and the recommended deployment configuration is `--workers 1`. This finding likely generalizes to other unified-memory SoC architectures (e.g., Qualcomm Snapdragon) where the memory hierarchy differs fundamentally from discrete CPU+GPU configurations.

8.5. Limitations and Future Work

Image data. The CIFAR and MNIST experiments use raw pixel vectors as input to a pre-trained sentence-transformer, which is not the intended input modality. The results demonstrate the method on a common benchmark; the strongest version of the approach would use a purpose-trained image encoder (e.g., CLIP, ResNet features) as the pre-computation stage.

Non-linear projection heads. PCA is a linear projection. A non-linear alternative (e.g., a UMAP projection to d^* dimensions) may better capture the manifold geometry for datasets where class boundaries are non-linearly separable in the principal subspace.

Adaptive τ selection. The threshold $\tau = 0.9$ is fixed across all experiments. Cross-validated τ selection could improve performance, particularly for datasets with heterogeneous per-class intrinsic dimensionality.

ManifoldModel scalability. The current $O(n^2)$ graph construction limits ManifoldModel to datasets below $\sim 10,000$ training points without subsampling. Approximate nearest-neighbor graph construction (e.g., via HNSW) would extend zero-parameter classification to larger corpora.

9. Conclusion

We have presented a system combining offline corpus pre-computation with manifold-aware classification, validated on four canonical benchmarks. The central empirical result is that intrinsic dimensionality estimation from pre-computed embeddings, combined with a linear

classification head in the estimated intrinsic subspace, consistently outperforms or matches standard deep networks at extreme parameter reduction. On CIFAR-10: Intrinsic Dim within 3.81 pp of Standard MLP at $2,387\times$ fewer parameters. On CIFAR-100: Intrinsic Dim wins outright — +23.8% relative accuracy at $184\times$ fewer parameters. On Digits: zero-parameter classification within 0.60 pp of a parameterized baseline. On MNIST: $106\times$ parameter reduction at 2.31 pp cost.

The offline pre-computation paradigm that makes this possible — computing all semantic structure at ingest time and freezing it in a hybrid knowledge graph — eliminates query-time model loading and inference while preserving full retrieval capability. The Pepys corpus case study demonstrates 14-minute full-pipeline execution on consumer CPU hardware, with sub-millisecond query latency thereafter.

We further document that on Apple Silicon unified-memory hardware, the CPU execution path outperforms explicit GPU (Metal) execution for small MLP workloads due to per-operation synchronization overhead — a finding with practical consequences for edge deployment of manifold-aware classifiers.

All code is publicly available.

Appendix A. Hyperparameter Summary

Table 9: Hyperparameters used in all benchmark experiments.

Parameter	Value
Optimizer	Adam
Learning rate	10^{-3}
Gradient clip norm	1.0
Loss	SparseCategoricalCrossentropy (from_logits=True)
Epochs	50
Batch size (MNIST, CIFAR)	128
Energy-retention threshold τ	0.9
k -NN graph neighbors k_{graph}	15
Voting neighbors k_{vote}	7
PCA k_{pca} (local PCA)	50
Discovery samples	500 (for local PCA dimensionality estimation)
ManifoldModel subsample (MNIST)	5,000 (stratified)
TF version	2.16.2
Hardware	Apple Silicon M-series (CPU execution path)
Python	3.10 (waverider), 3.12 (diary_kg)

Appendix B. Pipeline Configuration

Table 10: diary_kg pipeline configuration for Pepys corpus experiments.

Parameter	Value
Embedding model	<code>sentence-transformers/all-mpnet-base-v2</code>
Embedding dim	768 (float32)
Chunking strategy	<code>sentence_group</code>
Max chunk chars	512
Sentences per chunk	4
Max chunks per entry	3
Workers (embedding)	1 (optimal; see Table 1)
Batch size (embedding)	32
Structural index	SQLite with BM25 (FTS5)
Vector index	LanceDB (cosine ANN)

References

- Robert Calderbank, Sina Jafarpour, and Robert Schapire. Compressed learning: Universal sparse dimensionality reduction and learning in the measurement domain. 2009.
- Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.
- Elena Facco, Maria d’Errico, Alex Rodriguez, and Alessandro Laio. Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Scientific Reports*, 7(1):12140, 2017.
- Ronald A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.
- Jonathan Frankle and Michael Carlin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *International Conference on Learning Representations*, 2019.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning Workshop*, 2015.
- Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zhengyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems*, volume 2, 1990.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Elizaveta Levina and Peter J. Bickel. Maximum likelihood estimation of intrinsic dimension. *Advances in Neural Information Processing Systems*, 17, 2004.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- Phillip Pope, Chen Zhu, Ahmed Abdelkader, Micah Goldblum, and Tom Goldstein. The intrinsic dimension of images and its impact on learning. In *International Conference on Learning Representations*, 2021.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- Sam T. Roweis and Lawrence K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.
- Eric G. Suchanek. DocKG: A hybrid semantic and structural knowledge graph for document corpora. Technical report, Flux-Frontiers, 2026.
- Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.
- Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems*, volume 16, 2004.
- Xiaojin Zhu and Zoubin Ghahramani. Learning from labeled and unlabeled data with label propagation. 2002.